

Introduction to Computer Architecture

Chapter 3

Arithmetic for Computers

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

FLOATING POINT NUMBERS

Floating Point (1)

- What would be the output?

→ 64 bit floating-point data type

```
double a, b, c, d;  
a = 0.1;  
b = 0.7;  
c = 0.07;  
0.1 × 0.7 = 0.07  
d = a*b;  
  
if(c==d) printf("%f is equal to %f\n",c,d);  
else     printf("%f is NOT equal to %f\n",c,d);
```

25 bits 25 bits

0.070000 is NOT equal to 0.070000

Floating Point (2)

- Let's see how an integer number is represented

```
int i1 = 58; decimal  
printf("i1 = %d\n", i1);  
printf("i1 = 0x%X\n", i1);
```

```
i1 = 58  
i1 = 0x3A
```

0x0000003A

Floating Point (3)

- Let's see how a non-integer number is represented

```
float f1 = 58.0;
```

```
printf("f1 = %f\n", f1);  
printf("f1 = 0x%X\n", f1);
```

58 $\Rightarrow$ 111010
 $e=5 + 127 = 132$ 10000100

warning: format '%X' expects argument of type 'long unsigned int',
but argument 2 has type 'double'

```
f1 = 58.000000
```

```
f1 =
```

0 100|0010|011010
0X 4268 0000

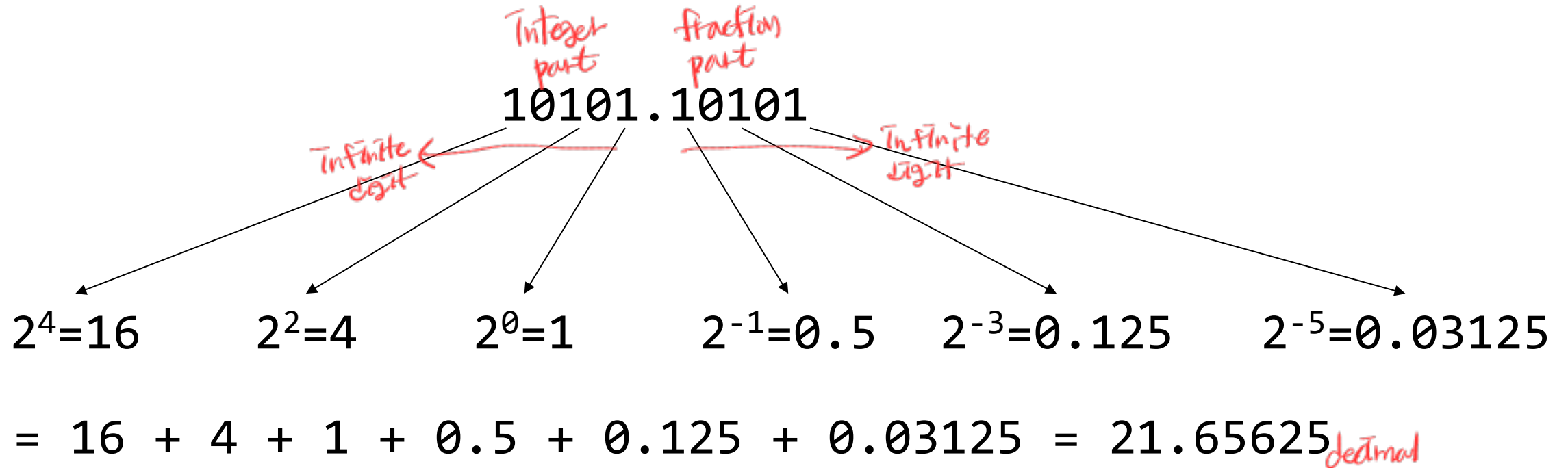
Floating Point (4)

- Let's see how a non-integer number is represented

```
float f1 = 58.0;  
unsigned int u1 = *((unsigned int *) &f1);  
  
printf("f1 = %f\n", f1);  
printf("u1 = 0x%X\n", u1);
```

```
f1 = 58.000000  
u1 = 0x42680000
```

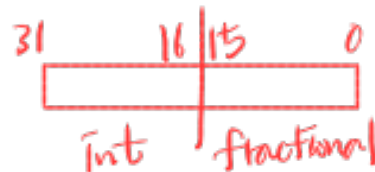
Representing Non-Integer Numbers in Binary



■ Question: How can we 'encode' the binary representation in computers?

❖ Fixed-point format?

- Fixed number of bits for integer / fractional parts
- Limited range!



2¹⁵-1 까지만 표현가능

Floating Point

■ Scientific notation (Significand + Magnitude)

❖ -2.34×10^{56}

❖ $+0.002 \times 10^{-4}$

❖ $+987.02 \times 10^9$

높은 값을
→ 다양하게
표현 가능

상수 x 32

~~$= -234 \times 10^{54}$~~ | 보라클

$= +0.2 \times 10^{-6}$

$= +9.8702 \times 10^{11}$

~~$= -0.234 \times 10^{57}$~~ | 보라값을

$= +2.0 \times 10^{-7}$

$= +98702 \times 10^7$

exponent

significand (or coefficient)

standard format

decimal이 아닌

1, 2, 3 ~ 8, 9로 끝

■ Normalized (or standard form)

❖ $1 \leq |\text{significand}| < 10$ decimal

Binary Representation

이진수 표현 방법

10101.10101

$$= 10101.10101 \times 2^0$$

$$= 1.010110101 \times 2^4$$

~~0.10101×2^{10}~~

1.0×2^9

↑
standard binary format

Range of the significand in binary normalized format:

$$1 \leq |\text{significand}| < 2$$

이진수

Binary number normalized format:

$$\pm 1.xxxxxxx_2 \times 2^{yyyy}$$

① sign bit ② mantissa ③ exponent

이진수 표현 방법

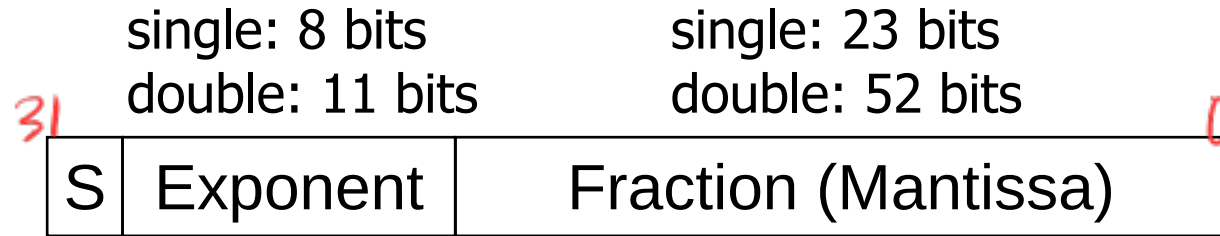
Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
 - ❖ Portability issues for scientific code
- Now almost universally adopted
- Single precision (32-bit) → float type in C
- Double precision (64-bit) → double type in C

31 0
/ 1
two's complement 2¹ 2⁰의 경우
→ 사실상 다른 방법이 없음

floating point의 경우
standard가 없었기 때문에
이런이 아닌 다른 문제

IEEE Floating-Point Format



$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: **sign** bit (0 $\Rightarrow$ non-negative, 1 $\Rightarrow$ negative)
positive
- Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
 - ❖ Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
 - ❖ Significand is **Fraction** with the “1.” restored
- **Exponent**: excess representation: actual exponent + Bias
 - ❖ Ensures exponent is unsigned
 - ❖ Single: Bias = 127; Double: Bias = 1023

1.011 $\times 2^3$
mantissa exponent

$$3 + 127 = 130$$

$$8\text{bit } 2^{8-1} - 1 = 127 \quad 11\text{bit } 2^{11-1} - 1 = 1023$$

Floating-Point Example

■ Represent -0.75

$$\diamond -0.75 = -\frac{3}{4} = -3 \times \frac{1}{4} = -11_2 \times 2^{-2} = -1.1_2 \times 2^{-1}$$

$$\diamond S = 1$$

$$\diamond \text{Fraction} = 1000\dots00_2$$

$$\diamond \text{Exponent representation} = -1 + \text{Bias}$$

$$\triangleright \text{Single: } -1 + 127 = 126 = 01111110_2$$

$$\triangleright \text{Double: } -1 + 1023 = 1022 = 01111111110_2$$

$$\text{Single: } \overset{B}{1} \overset{F}{0} \overset{4}{1} 1111101000\dots00$$

$$\text{Double: } \overset{B}{1} \overset{F}{0} \overset{E}{1} 111111101000\dots00$$

$$\overset{B}{1} \overset{F}{0} \overset{E}{1} 8 000\dots$$

23 bit (fraction part)

10000...0

$$-0.75 = -0.11_2$$

$$= -11_2 \times 2^{-2}$$

$$= -1.1_2 \times 2^{-1}$$

Exponent
Single: 126
Double: 1022

Single:

1 01111110 10000000

double

1 01111111110 10000000

Floating-Point Example

- What number is represented by the single-precision float

1|10000001|01000...00

- ❖ $S = 1$ $129 - 127 = 2$
- ❖ Fraction = $01000...00_2$
- ❖ Exponent representation = $10000001_2 = 129$

- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$
 $= (-1) \times 1.25 \times 2^2$
 $= -5.0$

$$e + \text{Bias} = 129$$

127

$$\downarrow$$
$$2$$

$$-1.01 \times 2^2$$
$$= -101_2$$
$$= -5$$

Single-Precision Range

- Exponent representations “00000000” and “11111111” are reserved

- Smallest value

Exponent representation: 00000001 $\Rightarrow$ actual exponent = $1 - 127 = -126$

Fraction: 000...00 $\Rightarrow$ significand = 1.0

$\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$

Integer value range $-2^{31} \sim 2^{31}-1$

가장 작은 값에 $1.0 \times 2^{-126} =$

- Largest value

Exponent representation: 11111110 $\Rightarrow$ actual exponent = $254 - 127 = +127$

Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0

$\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

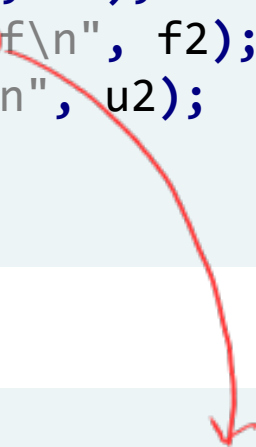
$1.111...111 \times 2^{127}$
 $\frac{2^3}{2} = 2$

Floating Point (5)

- What would be the output?

```
float f2 = -0.1;  
unsigned int u2 = *((unsigned int *) &f2);  
  
printf("f2 = %f\n", f2);  
printf("f2 = %3.20f\n", f2);  
printf("u2 = 0x%X\n", u2);
```

```
f2 = -0.100000  
f2 = -0.10000000149011611938  
u2 = 0xBDCCCCCD
```



Integer보다 범위는 넓지만
precision이 한계가 있다

Limited Precision

■ Let's represent 0.1 in single-precision Floating Point

$$0.1 = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{512} + \frac{1}{4096} \dots$$

$2^{-4} \quad 2^{-5} \quad 2^{-8} \quad 2^{-9}$

$$\sum \frac{1}{2^n} = 0.000110011\dots = 1.10011\dots \times 2^{-4}$$

Mantissa value는
original significand와
값이 다를 수 있음

$$\text{Exponent} = -4 \rightarrow -4 + 127 = 123 = 01111011_2$$

$$\text{Significand} = 1.10011001100110011001100110011001100110011001100\dots$$

계속 반복

$$\text{Mantissa} = 10011001100110011001100110011001$$

round up

$$\begin{matrix} \text{sign} & \text{exponent} & \text{mantissa} \\ 0 & 01111011 & 10011001100110011001100110011001 \end{matrix}$$

$$0011 \ 1101 \ 1100 \ 1100 \ 1100 \ 1100 \ 1100 \ 1101 \rightarrow 0x3DCCCCCD$$

$$0x3DCCCCCD \rightarrow 0.100000001490116119384765625$$

0.1와는 조금 다름
rounding에 따라 값이 달라질
round down인 경우 0.1보다 작아질

0011
↓
1011

0.1
-0.1이면?
0x3DCCCCCD (other slide)

Floating-Point Addition

- Consider a 4-digit decimal example

❖ $9.999 \times 10^1 + 1.610 \times 10^{-1+2=1}$

1. Align decimal points

❖ Shift number with smaller exponent

❖ $9.999 \times 10^1 + 0.016 \times 10^1$

2. Add significands

❖ $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^{1+1}$

3. Normalize result & check for over/underflow

❖ 1.0015×10^2 ← standard

4. Round and renormalize if necessary

❖ 1.002×10^2

4 digit

rounding
renormalization 필요할 때가 있음
round operation이 없을 때가 있음

Integer는 같은 자리의 배를 더하면 끝
→ 예외선 exponent에 따라 배가 달라
의미하는 digit이 다를 수 있음
→ exponent를 먼저 맞춰주어야 함

→ standard format이 아님

9.9999×10^2

↓ ← rounding을 한 결과:
standard format X

10.000×10^2

↓ ← renormalization

1.000×10^3

Floating-Point Addition

■ Now consider a 4-digit binary example

❖ $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ ($0.5 + -0.4375$)

1. Align binary points

❖ Shift number with smaller exponent

❖ $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$

2. Add significands

❖ $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$

3. Normalize result & check for over/underflow

❖ $1.000_2 \times 2^{-4}$, with no over/underflow

4. Round and renormalize if necessary

❖ $1.000_2 \times 2^{-4}$ (no change) = 0.0625

shift left,
exponent decrement

$2^0_2 \quad 2^2_2 \quad 2^3_2 \quad 2^4_2$

$$\begin{array}{r} 1.000_2 \\ - 0.111_2 \\ \hline 0.001_2 \end{array}$$

$\frac{1}{2} \quad \frac{7}{8} \quad \frac{1}{8}$

FP Adder Hardware

- Much more complex than an integer adder
- Doing it in one clock cycle would take too long
 - ❖ Much longer than integer operations
- FP adder usually takes several cycles
 - ❖ Can be pipelined

Integer의 비해
floating point의
cycle이 더 길다

더러 cycle 사용

FP Adder Hardware

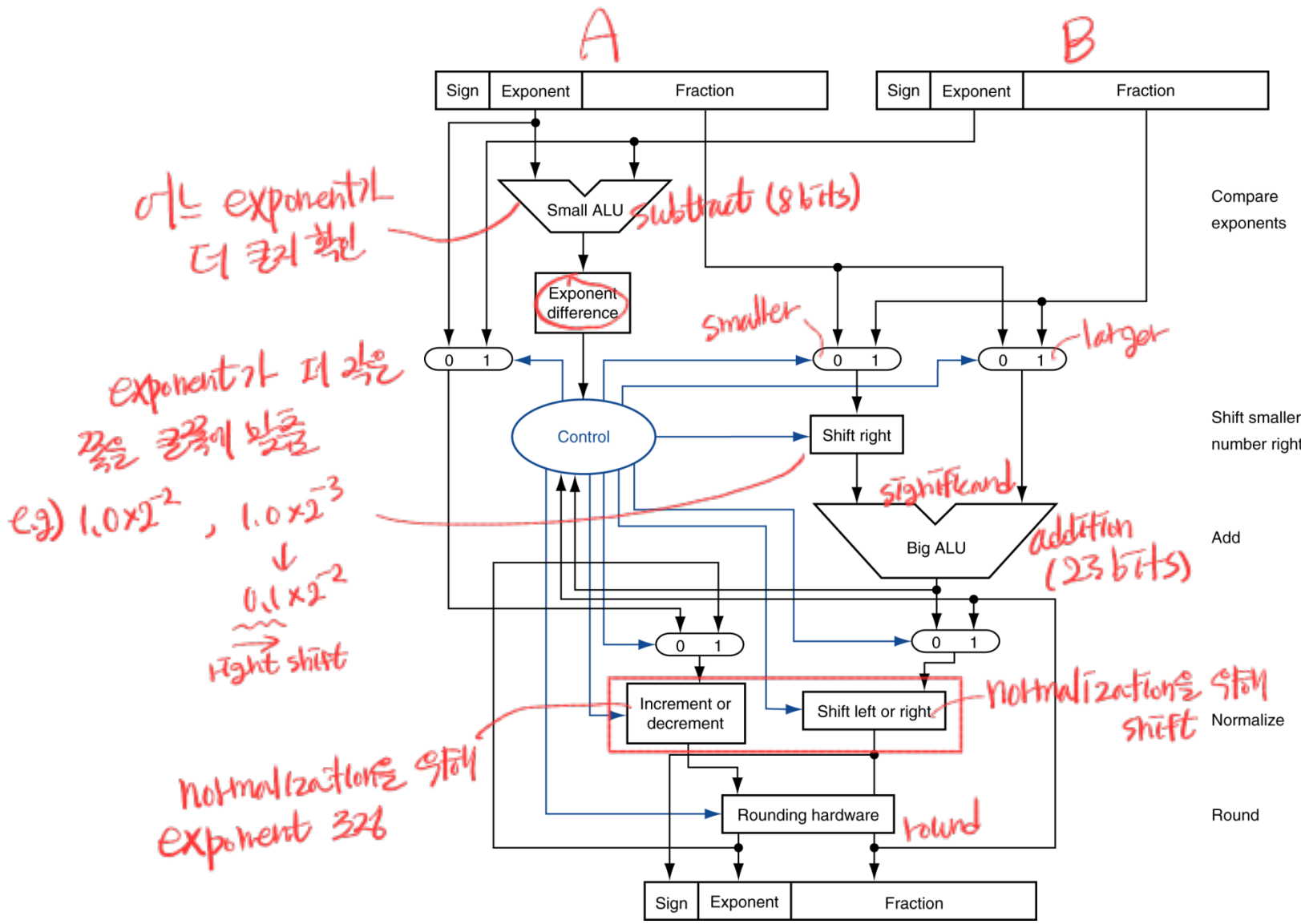
별명 기기는 floating point 연산은
각종이러한 일들 (연산이 복잡하기 때문)

→ CPU가 integer operation만 가능하고
Emulator를 통해 floating point
operation을 소프트웨어로 처리 가능

→ 그러나 너무 느림,
대부분이 하드웨어적으로 처리하는 게
좋음.

floating point는 기본적으로
integer보다 더 많은 연산 사용
특히 그래픽의 GPU에서 많이 사용

Standard format
이러한 normalization 방법



Step 1

Step 2

Step 3

Step 4

Why use Bias in Exponent?

$$\times 2^{-3} + 127 \Rightarrow 124$$

■ Consider two numbers: 2.5 and 0.75

- ❖ $2.5 = 10.1_2 = 1.01_2 \times 2^1$
- ❖ $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$

■ If no bias is used (2's complement representation for exponent),

❖ 2.5

$$1.01 = 1.25 \times 2 = 2.5$$

- Mantissa: $01000...00_2$ Exponent representation: $1 = 00000001_2$
- Binary representation: $00000000101000...00_2 = 0x00A00000$

❖ 0.75

- Mantissa: $10000...00_2$ Exponent representation: $-1 = 11111111_2$
- Binary representation: $01111111110000...00_2 = 0x7FC00000$

Integer로 나타낼
아래의 더 커보일

Which one is bigger?

Why use Bias in Exponent?

■ Consider two numbers: 2.5 and 0.75

- ❖ $2.5 = 10.1_2 = 1.01_2 \times 2^1$
- ❖ $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$

floating point의 크기를 비교할 때는
Integer처럼 bit만 보고 판단 가능

■ If bias (127) is used,

❖ 2.5

- Mantissa: $01000...00_2$ Exponent representation : $1 + 127 = 10000000_2$
- Binary representation: $01000000001000...00_2 = 0x40200000$

❖ 0.75

- Mantissa: $10000...00_2$ Exponent representation : $-1 + 127 = 01111110_2$
- Binary representation: $00111111010000...00_2 = 0x3F400000$

✓ 각 bit의 크기를 비교할 수 있고, 음수는 0보다 작아진다

Which one is bigger?

Infinites and NaNs *special case*

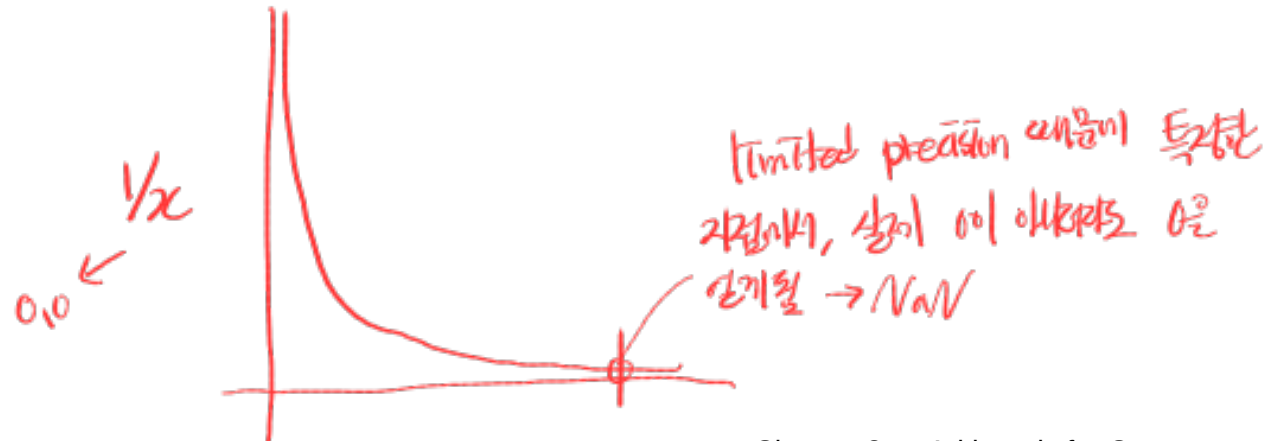
- *✓* Exponent representation = 11111111, *all 0* Fraction = 00000...00
 - ❖ $\pm$ Infinity
 - ❖ Can be used in subsequent calculations, avoiding need for overflow check

*0 1111 1111 000...000 $\Rightarrow$ 0x7FF80000 $\leftarrow +\infty$
1 1111 1111 000...000 $\Rightarrow$ 0xFF800000 $\leftarrow -\infty$*

- *✓* Exponent = 11111111, Fraction $\neq$ 00000...00

- ❖ Not-a-Number (NaN)
- ❖ Indicates illegal or undefined result
 - e.g., 0.0 / 0.0

*0 1111 1111 111...111 $\rightarrow$ NaN
0 1111 1111 000...001 $\rightarrow$ NaN*



Denormalized Numbers *2⁻¹²⁶ 이하 값을 위한*

- What would be the smallest number without the denormalized numbers?
 - ❖ Suppose we can use “00000000” as a normal exponent: $1.0 \times 2^{-127} = 5.87... \times 10^{-39}$
⇒ not enough
 - ❖ May not be small enough for some cases..
 - e.g., When $0 < |x - y| < 2^{-127}$, calculating $\frac{1}{x-y}$ would result in a divide by zero.
- How can we represent numbers smaller than 2^{-127} ?
 - ❖ First, reserve the exponent “00000000” as a special case
 - Now, the smallest number under the normalized form is 2^{-126}
 - ❖ For numbers smaller than 1.0×2^{-126} , use the “denormalized number” format.

$$\begin{array}{l} 1. \boxed{\text{fraction/mantissa}} \leftarrow 2^{-126} \\ 0. \boxed{\phantom{\text{fraction/mantissa}}} \leftarrow 2^{-126} \\ \quad \downarrow \\ 0.01 \times 2^{-126} = 2^{-128} \end{array}$$

Denormalized Numbers

- Exponent = 000...0

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-(\text{Bias}-1)}$$

$\pm 0. \boxed{} \times 2^{-126}$

- Denormalized numbers do not assume the significand starts with “1.0”.

- ❖ The integer part of the significand is “0.”
- ❖ The magnitude is always $\times 2^{-126}$

- Denormalized number example: $1.01_2 \times 2^{-130} = 0.\boxed{000101}_2 \times 2^{-126}$

- ❖ Exponent = 00000000
- ❖ Mantissa = 00010100...00₂

$000 \overbrace{101 \dots 000}^{23}$
20 bits만 사용됨 → precision이 더 limited됨

- The effective precision is decreased compared to the normalized numbers. precision ↓

New FP Formats

machine learning을 위한 (dataset의 대용량, precision을 크게 줄임)

■ Sign ■ Exponent ■ Mantissa

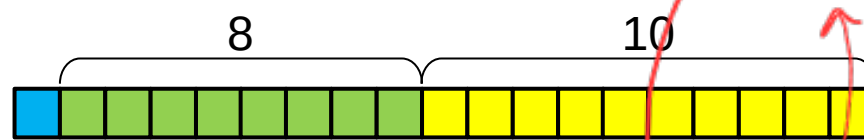
■ **FP32**
single
4 byte



1967년에
align하지 않음

"Single Precision"

■ **TF32**
2 byte

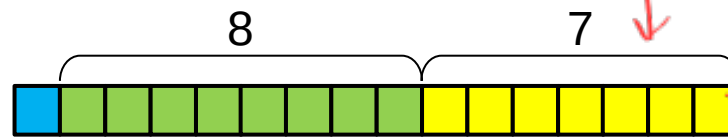


↑ 1967년 0.32로 (BF16 → FP32)

"Tensor Float 32"

FP16과 같은 precision, FP32로 convert 가능

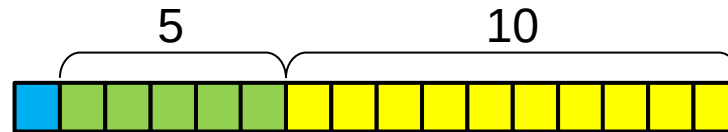
■ **BF16**



"Brain Float 16"

→ FP32로 같은 exponent, convert 가능

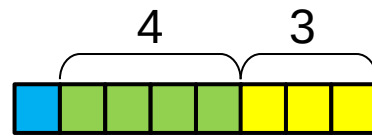
■ **FP16**
2 byte



"Half Precision"

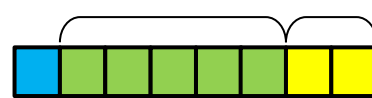
narrow range,
precision,
simple

■ **FP8 (E4M3)**



상황이 다르면 선택

■ **FP8 (E5M2)**



특정 machine의 성능